Лабораторная работа №3.
Работа с Git

Цели лабораторной работы

1. Изучить основы версионирования, основные команды и возможности Git;
2. Изучить принципы командной работы с Git.

Задачи к лабораторной работе

1. Изучить методические материалы к лабораторной работе;
2. Установить Git, если не установлен;
3. Изучить работу одиночного репозитория;
4. Изучить командную работу с репозиторием, принцип центрального репозитория.

Методические материалы

Основы версионирования

Задача версионирования документов появилась довольно давно и классически решалась за счёт специальных имён файлов:

- Уникальное имя для каждой версии (или UUID);
- Добавление временной метки в имя файла;
- Добавление новых постфиксов для каждой новой версии;
- Специализированные форматы, поддерживающие историю изменений.

Однако, в задачах написания кода все эти приёмы неприменимы, так как имена файлов и их расположение имеют важную роль для различных инструментов разработки, библиотек, фреймворков и т.д.

Таким образом был создан отдельный класс программ: система контроля версий (Version Control System, VCS).

Система контроля версий (СКВ) — программа или система, фиксирующая изменения в файле или файлах, позволяющая зафиксировать текущую версию и позволяющая вернуться позже к определённой версии. Зафиксированная версия обычно сопровождается метаданными о создателе версии, дате и времени фиксации, и отличиях данной версии от предыдущей.

Системы контроля версий можно разделить на локальные, централизованные и распределённые:

- Локальные — хранят историю изменений и управляют ими на машине пользователя. Данные системы не применимы для командной работы. Могут быть встроены в другое приложение.
- Централизованные — появились как способ решения проблемы взаимодействия нескольких разработчиков в одном проекте. На специальном сервере хранятся все версии файлов. К этому серверу обращаются клиенты, которые получают отдельные файлы из централизованного хранилища. У такого подхода есть ряд минусов:
    - Единая точка отказа — в случае отказа сервера системы контроля версий, история версий становится недоступной.
    - Отсутствие локальной истории — централизованные системы обычно не имеют копии истории локально, только редактируемые файлы. Все операции работы с репозиторием СКВ являются сетевыми.
    - Сложное решение конфликтов — Поскольку история хранится удалённо, процесс решения конфликта между разработчиками усложнён.
- Распределённые — полная копия репозитория хранится у каждого участника. Каждый участник работает с локальной копией репозитория, время от времени синхронизируя его с удалённым. В данном случае каждая копия репозитория может считаться резервной. Операции с СКВ выполняются локально, что упрощает совместную работу, снижая число конфликтов и упрощая процесс их решения. Удалённый сервер решает задачу исключительно хранения копии репозитория, а не единой точки работы с репозиторием.

Система контроля версий Git

Git — бесплатная распределенная система контроля версий с открытым исходным кодом. Она была разработана сообществом разработчиков ядра Linux в 2005 году как замена проприетарной СКВ BitKeeper, ранее использовавшейся при разработке ядра Linux.

Основное отличие Git от других систем — подход к работе с данными. Большинство СКВ хранят информацию в виде набора файлов и изменений (дельт), сделанных в каждом файле, по времени (контроль версий, основанный на различиях). Git же сохраняет снимки файловой системы. Для оптимизации этого процесса, существуют два механизма:

1. Если файл не был изменён, но в новый снимок помещается ссылка на предыдущую версию файла.
2. Если в системе становится слишком много похожих больших файлов, включается упаковка этих файлов на основе дельта-кодирования.

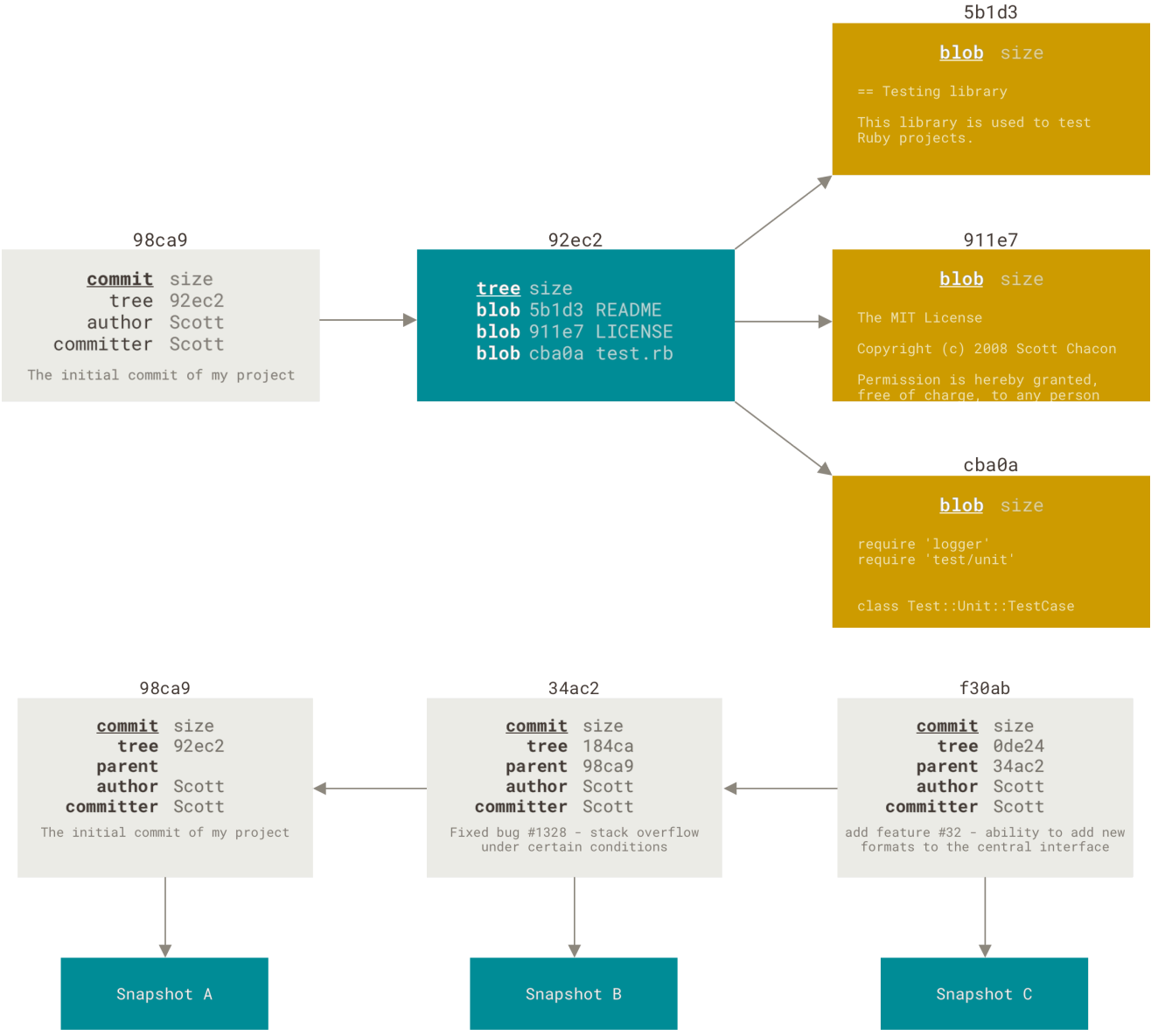
Система хранения фалов в Git

Все данные репозитория хранятся в служебной папке `.git`. Вы можете подробно изучить её структуру, однако же нас сейчас интересует только две папки: `objects` и `refs`.

Папка `objects` непосредственно хранит данные нашего репозитория. Существуют объекты следующего типа:

1. Blob (блоб, объект) — это непосредственно содержимое файла определённой версии, который был добавлен в репозиторий. Имя данного файла формируется как SHA-1 от специального заголовка и содержимого оригинального файла. Заметьте, *оригинальное* имя файла, права доступа, владелец на данном этапе безвозвратно утеряны.
2. Tree (дерево) — этот тип файла предназначен для хранения информации о файлах и подобен папкам в системе UNIX. Его имя также формируется с помощью SHA-1. Он хранит:
    - Права оригинального файла по одной из допустимых масок (`git` не сохраняет оригинальные права как есть, учитывается только наличие прав на исполнение и тип файла)
    - Тип объекта (blob или tree, то есть типы самого git)
    - Имя файла в папке `objects` (первые два символа отводятся под название подпапки, остальные название файла в этой подпапке)
    - Оригинальное имя файла
3. Commit (коммит) — данный тип файла описывает снимок состояния и обязательно ссылается на объект `tree`. Также имеет метаданные.
4. Tag (тег) — объект для создания аннотированных тегов, то есть, тегов с дополнительными метаданными.

На рисунках ниже схематично представлены связи между объектами.



Для того чтобы мы могли ориентироваться в наших коммитах, создавать ветки и теги, помечать удалённые ветки, нужны ссылки. Ссылки хранятся в папке `refs` в виде простых файлов, которые хранят хеш коммита:

- `heads` — Здесь хранятся ветки, то есть ссылки на последний коммит каждой ветки.
- `remotes` — Здесь хранятся последние известные ветки удалённого репозитория. Стоит отметить, что несмотря на название, коммиты, на которые эти ссылки сделаны, также присутствуют локально в папке `objects`.
- `tags` — Здесь хранятся простые и аннотированные теги.

Важно Как вы могли заметить, корневым объектом так или иначе является `blob`. Это приводит к интересному эффекту: пустые каталоги *невидимы* для системы git. Для сохранения структуры каталогов, негласно применяют файл с названием `.gitkeep`, создаваемый в пустой директории.

Состояния рабочего каталога

Рабочим каталогом является корень репозитория. Служебная папка `.git` также лежит в корне, однако по соглашению UNIX, файлы, начинающиеся с `.` не видны в команде `ls` без флага `-a` (all). В комит включается снимок именно рабочего каталога. Ссылка на текущий комит, с которым связано состояние рабочего каталога, хранится в файле `.git/HEAD`

У Git есть три основных состояния и одно дополнительное, в которых могут находиться файлы:

- Изменён (modified) — К изменённым относятся файлы, которые поменялись с последней фиксации, но не были добавлены в индекс (подготовительный этап перед следующей фиксацией).
- Индексирован (staged) — Этот статус принимают изменённые файлы, которые были подготовлены к фиксации.
- Зафиксирован (committed) — Этот статус означает, что файл был сохранён в папку `objects` и на него ссылается коммит (напрямую или через деревья).
- Не отслеживается (no tracked) — Этот файл означает, что файл ни разу не добавлялся в систему git, а потому не отслеживается. Git игнорирует такие файлы и никак с ними не взаимодействует. При переключении текущего коммита рабочего каталога (файл `HEAD`), данные файлы затронуты не будут.

Рабочий каталог (working tree) является единственной видимой и доступной для редактирования файлов секцией общего рабочего пространства git. Область индексирования (staging area, индекс) и область хранения коммитов являются виртуальными и фактически хранятся в `.git` .

Ветвление в Git

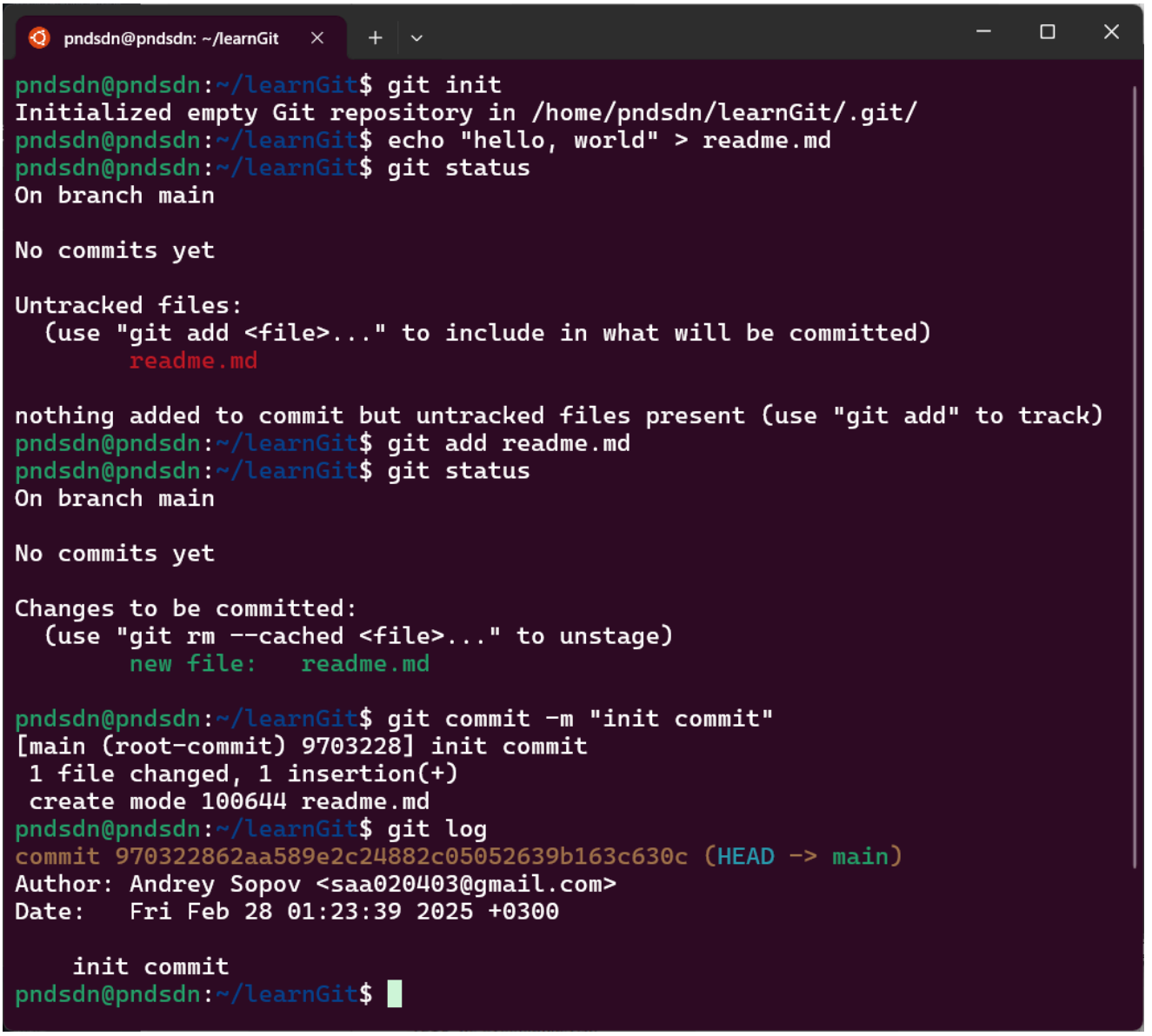
Механизм ветвления в системах контроля версий позволяет вести независимо параллельную разработку, поддерживая одновременно несколько версий проекта.

Ветки

Ветка в Git — это перемещаемый указатель (ссылка) на коммит. По умолчанию, имя основной ветки в Git — master, хотя на данный момент используются и другие варианты. HEAD — указатель на текущую ветку или коммит, на котором мы в данный момент находимся. В Git существует понятие *detach HEAD* (отсоединённый HEAD), это означает, что коммит, на который ссылается HEAD, не может быть

родителем нового, то есть, мы не можем от него зафиксировать новую версию. Это часть системы защиты от неправильной работы с репозиторием.

Разберем на примере.



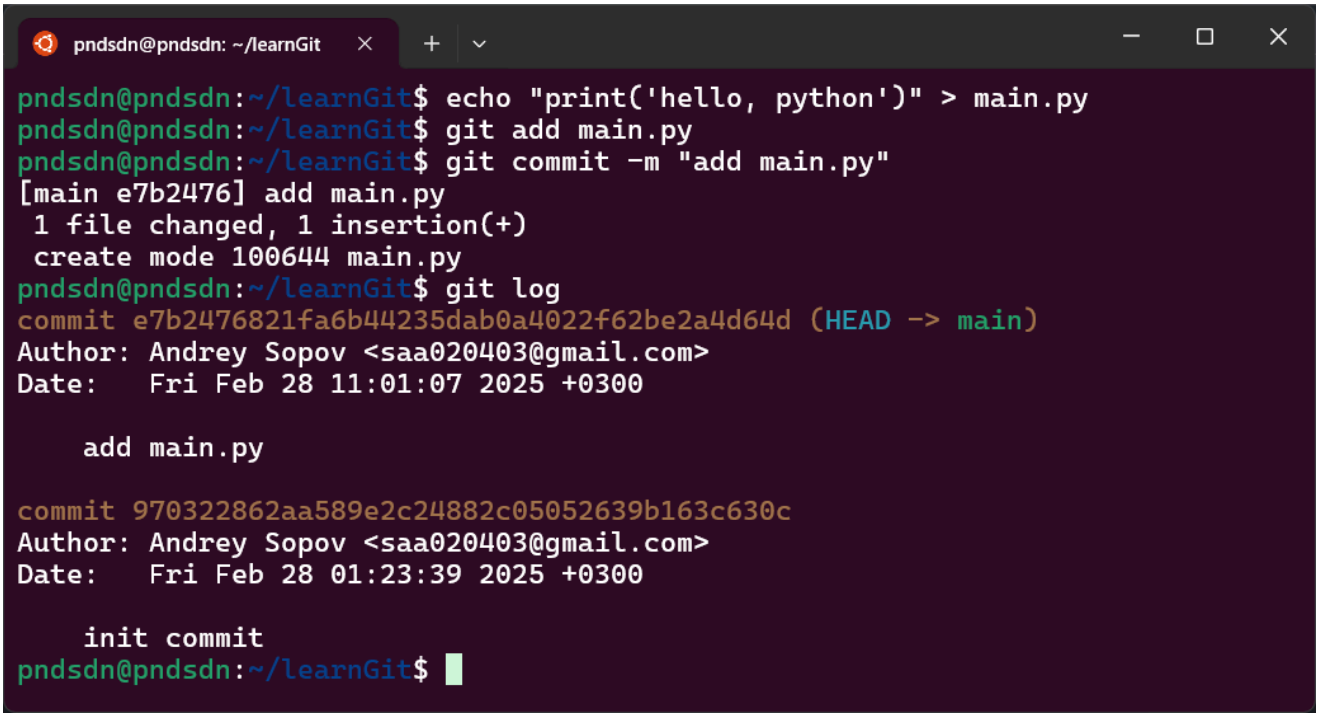
На рисунке выше:

1. Создан локальный репозиторий (`git init`);
2. Файл readme.md с содержимым “hello, world”;
3. Была получена информация о текущем состоянии репозитория (`git status`). Вывод показывает, что в рабочей копии находится файл readme.md;
4. Добавляем файл в индекс (`git add readme.md`), тем самым говорим, что хотим зафиксировать его изменения;
5. Проверяем статус ещё раз и видим, что теперь файл помечен как готовый к фиксации (коммиту);
6. Коммитим изменения с сообщением “init commit”(`git commit -m 'init commit'`). Если не указать флаг -m с сообщением, то откроется редактор по

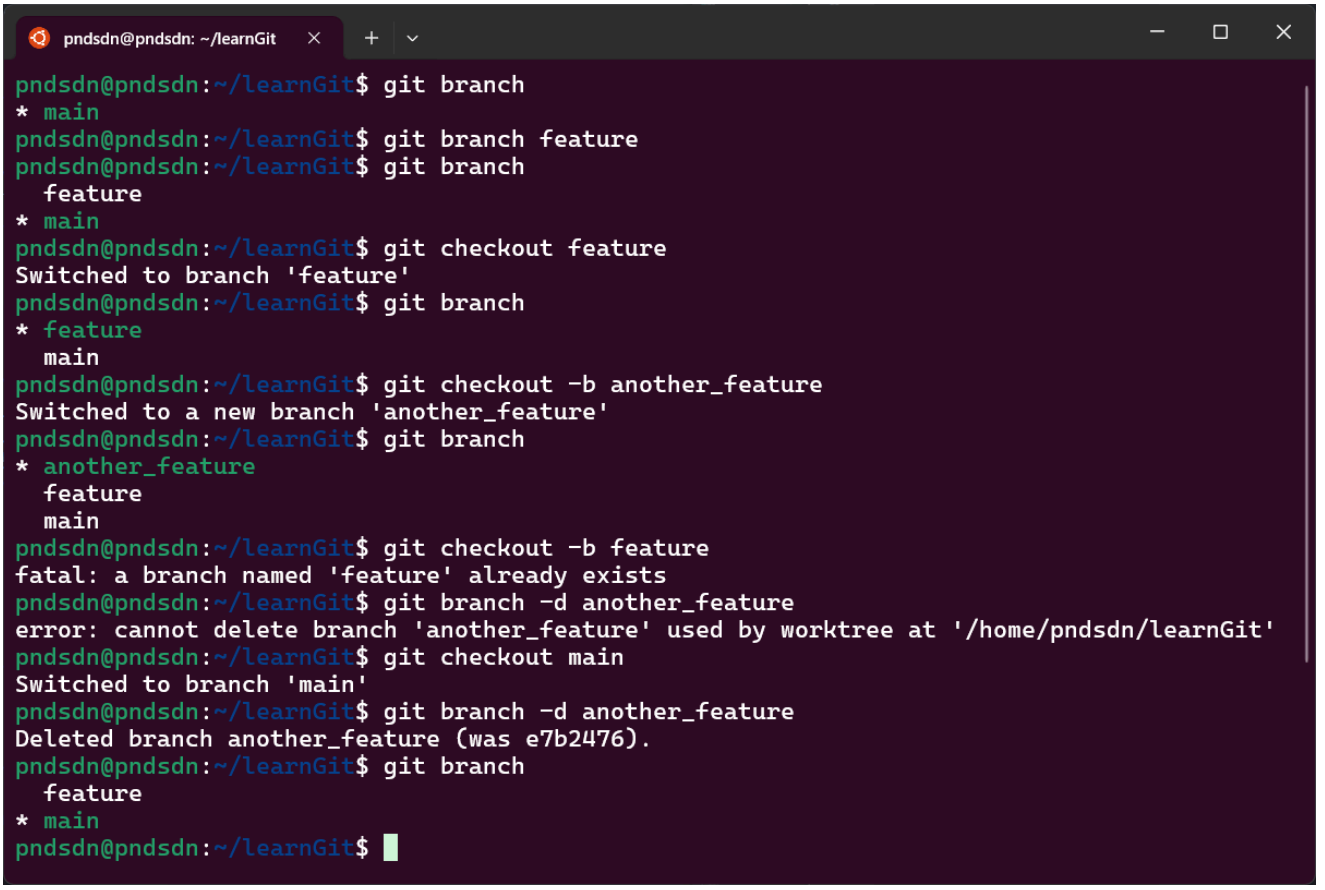
умолчанию (к примеру, *vim* или *nano*) и будет предложено ввести комментарий;

7. Последним действием проверяем историю коммитов (`git log`). Сейчас в ней находится только что созданный коммит.

Добавим файл main.py, закоммитим и посмотрим на историю:

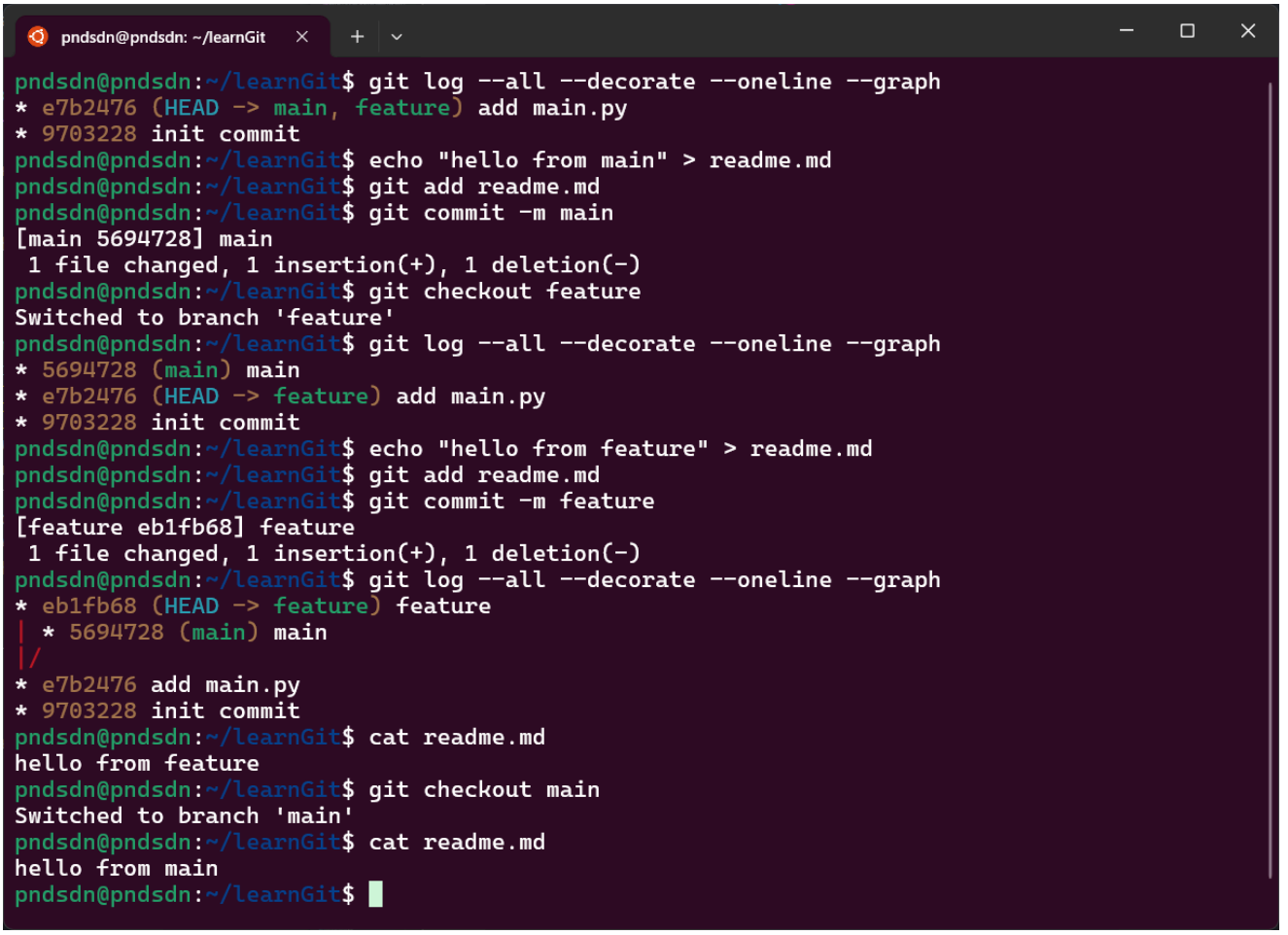


Обратите внимание, что указатель на ветку *main* переместился на последний коммит. А также **HEAD** указывает на ветку *main* (HEAD → main). Рассмотрим процесс создания, переключения и удаления веток:



1. Вывели список существующих веток (`git branch`). На данный момент у нас есть только одна ветка main. Зелёный шрифт и знак * означают указатель **HEAD**;
2. Создали новую ветку feature (`git branch feature`);
3. Обратите внимание, что HEAD по-прежнему указывает на *main*. Если на этой стадии сделать коммит, то вперёд продвинется указатель *main*, а не *feature*;
4. Переключились на ветку *feature* командой `git checkout feature` ;
5. Теперь вывод списка сообщает, что указатель **HEAD** переместился на ветку *feature*;
6. С помощью команды `git checkout -b another_feature` создается ветка *another_feature* и указатель **HEAD** перемещается на неё;
7. Однако вызов команды `git checkout -b feature` приводит к ошибке, так как ветка с таким именем уже существует;
8. Удаление ветки делается с помощью флага `-d` команды `git branch` . Но попытка удалить ветку, на которой вы находитесь, приведёт к ошибке. Поэтому сначала переключаемся на другую ветку, а только после этого удаляем.

Как говорилось ранее, ветки позволяют поддерживать несколько версий проекта. Продемонстрируем это на примере:



1. Команда `git log --all --decorate --oneline --graph` выводит историю коммитов в виде графа всех веток с коротким комментарием в одну строку;
2. В ветке *main* обновим файл `readme.md` и зафиксируем изменения;
3. Переключимся на ветку *feature*. Обратите внимание, что она указывает не на последний коммит, а на тот, на котором она находилась до этого (хеш `e7b2476`);
4. Опять поменяем содержимое `readme.md` и закоммитим;
5. Теперь мы видим, как наш проект разделился на две независимые ветки, с общим родителем — коммитом `e7b2476`;
6. Проверили, что файл readme.md имеет две версии с изменениями в соответствующих ветках.

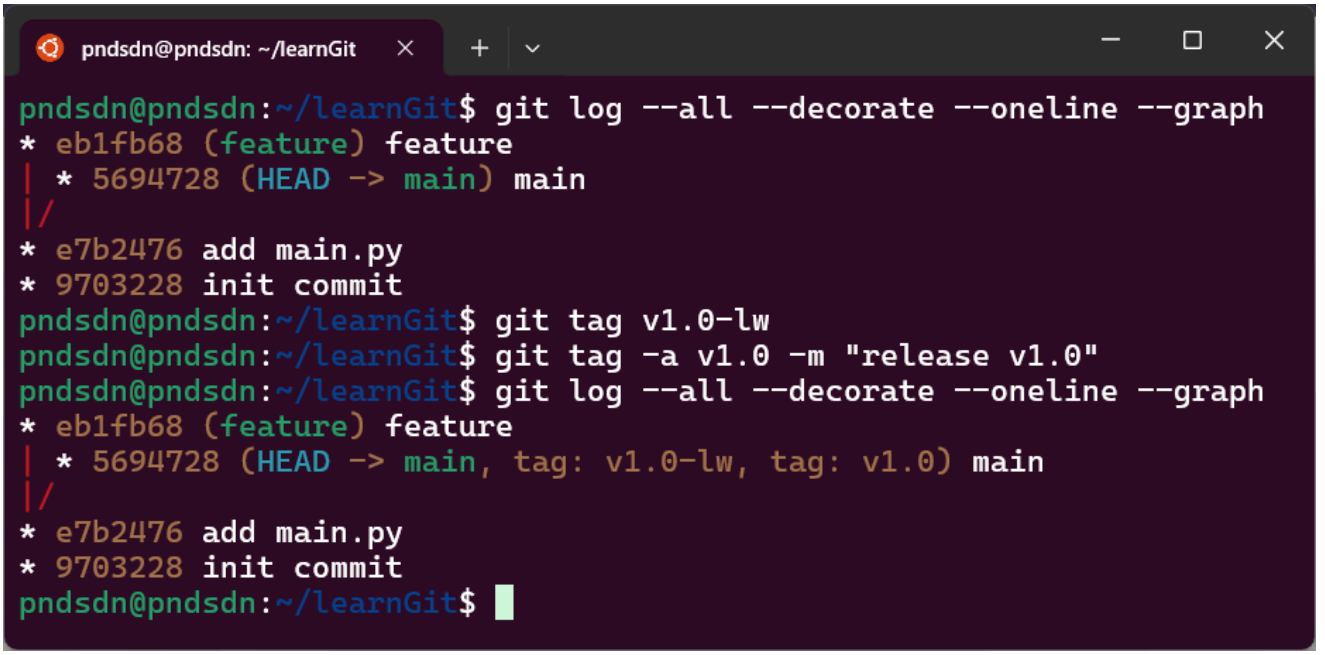
Теги

Теги в Git — это способ помечать определённые коммиты. Создание тега не приводит к созданию ветки. Как правило, они используются для отметок версий приложения (v1.0.0 и т.п.).

Git поддерживает два вида тегов:

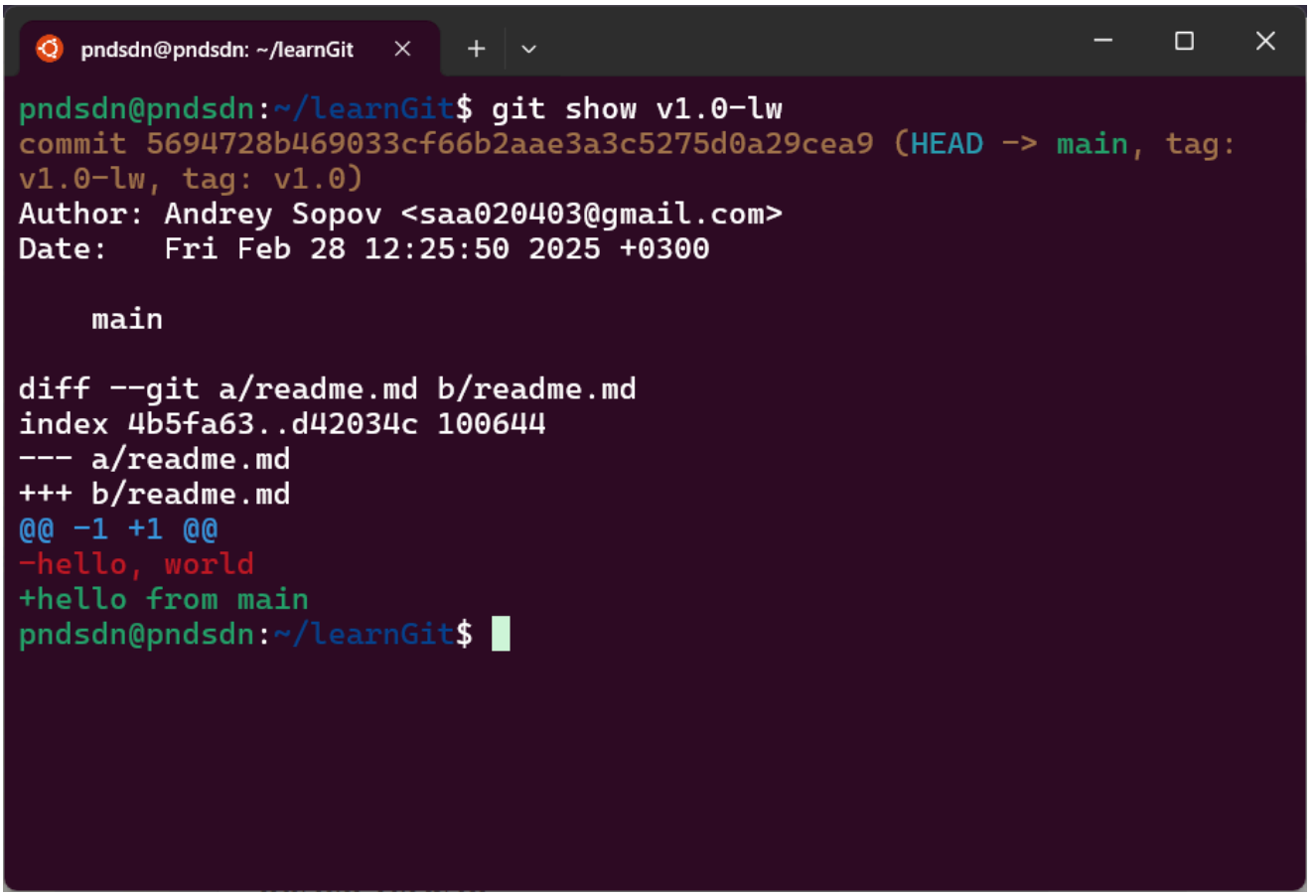
- Легковесные — указатель на определенный коммит, который не изменяется. Указывают напрямую на объект коммита и тем похожи на указатель ветки.
- Аннотированные — указатель не на определённый коммит, а на полноценный объект тега, который имеет ссылку на коммит, имя автора, его e-mail, дату создания и комментарий. Обычно рекомендуется создавать аннотированные теги.

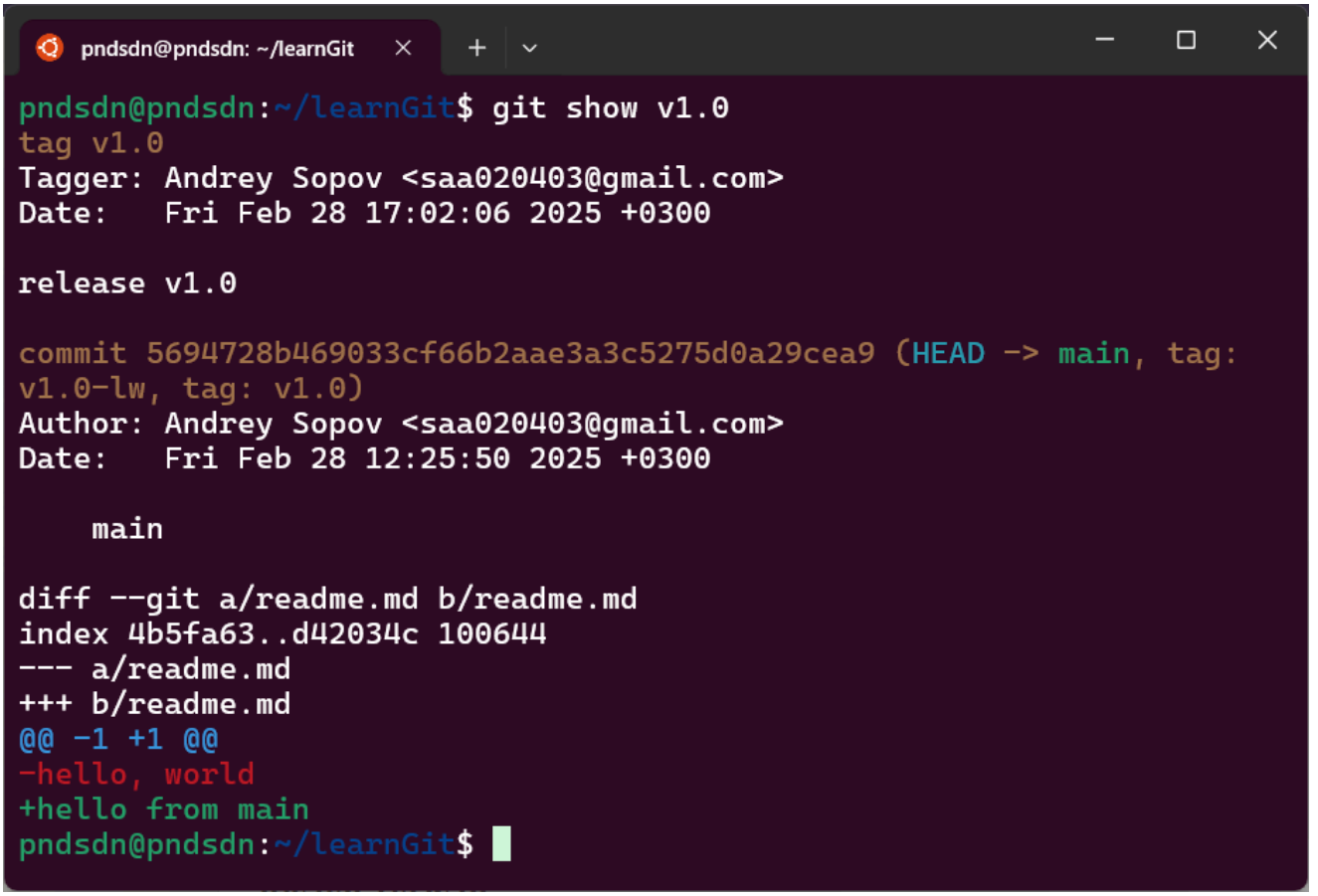
Создадим два тега: легковесный и аннотированный.



1. Для создания легковесного тега надо вызвать команду `git tag <tag_name>` без всяких параметров;
2. Для создания аннотированного тега необходимо передать параметр `-a`. Если не указать параметр `-m` с сообщением, то откроется текстовый редактор с предложением написать комментарий.

Сравним результаты вывода информации о тегах:





1. В первом случае видно, что тег просто ссылается на коммит, никакой дополнительной информации мы не видим;

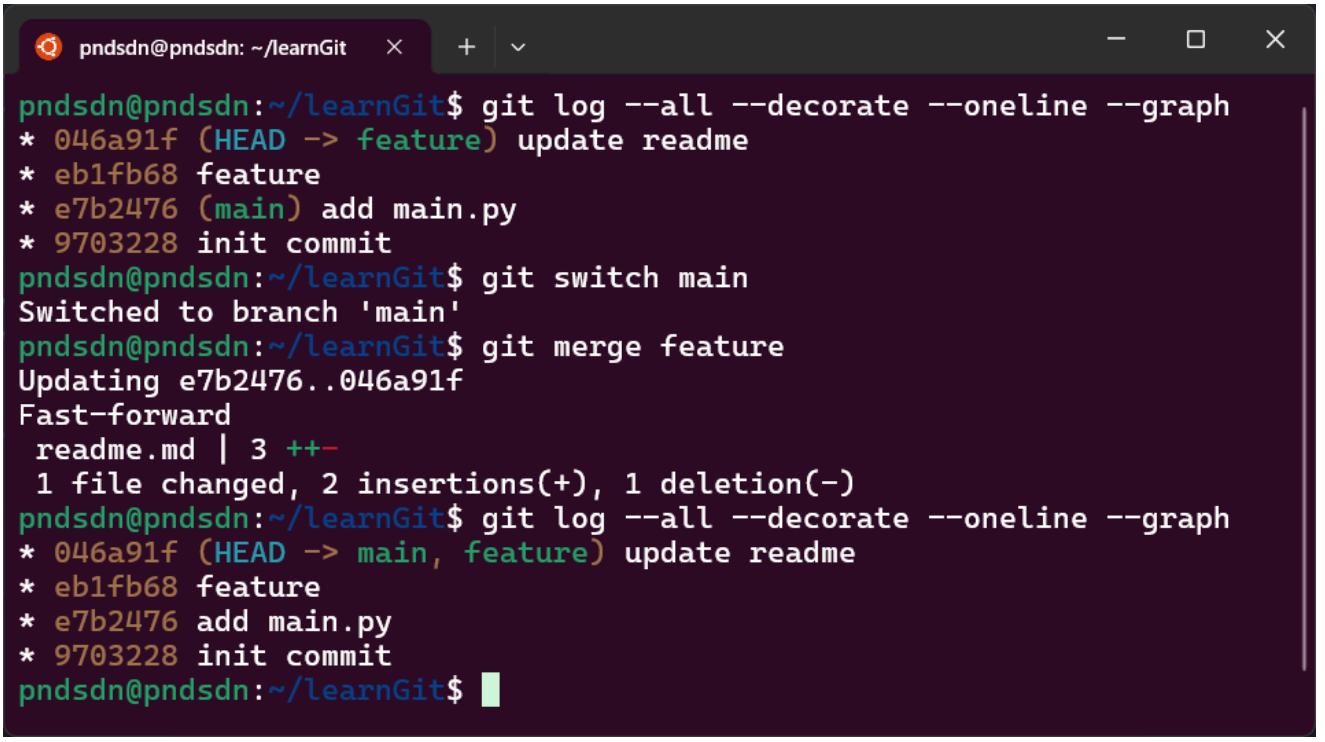
2. Во втором случае, помимо информации о коммите, видно кто и когда создал тег, а также комментарий к нему.

Слияние и перебазирование веток

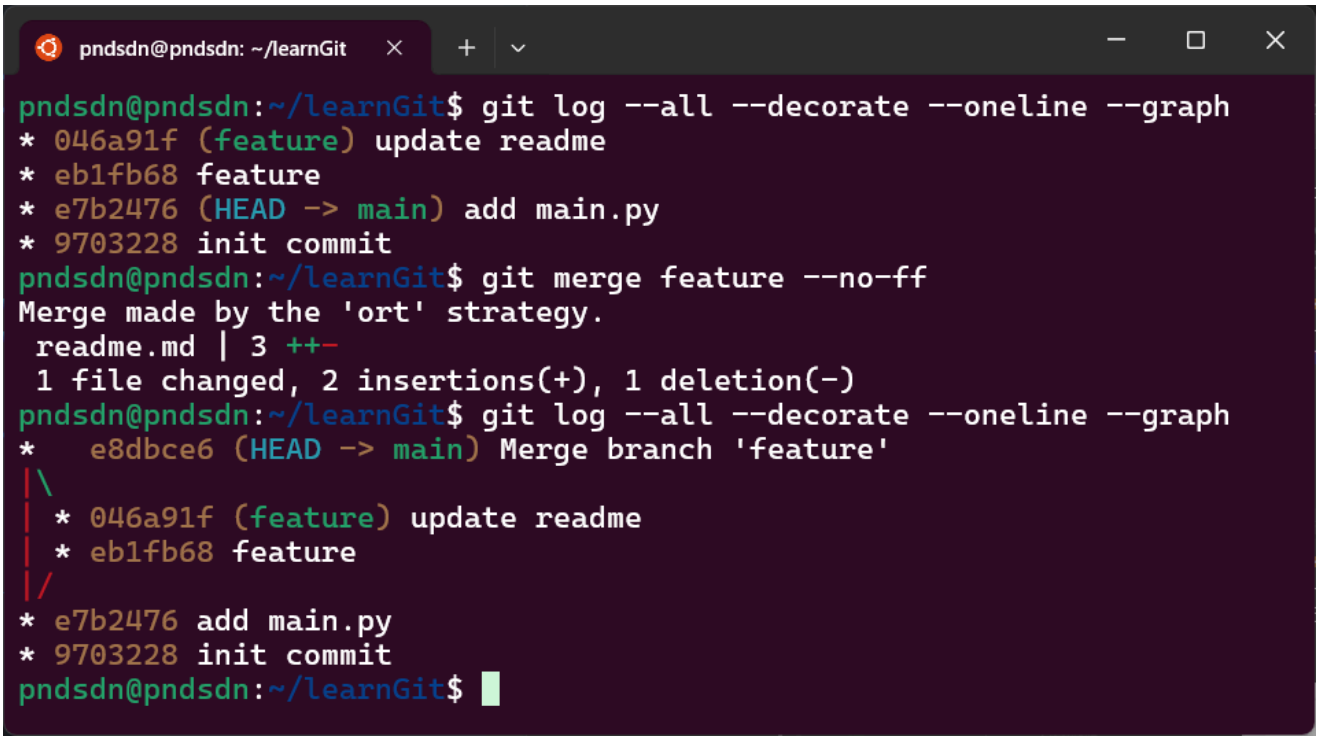
В процессе работы у вас может появиться потребность объединить код из разных веток: к примеру, в основную ветку вашего проекта надо добавить функционал из ветки *feature*. Git предоставляет несколько способов, как это можно сделать.

Fast-forward

В приведённом ниже примере ветка *feature* опережает ветку *main* на два коммита, то есть ветка *main* не имеет параллельной истории версий, а граф имеет линейную структуру. В такой ситуации достаточно перенести указатель вперёд (fast-forward):

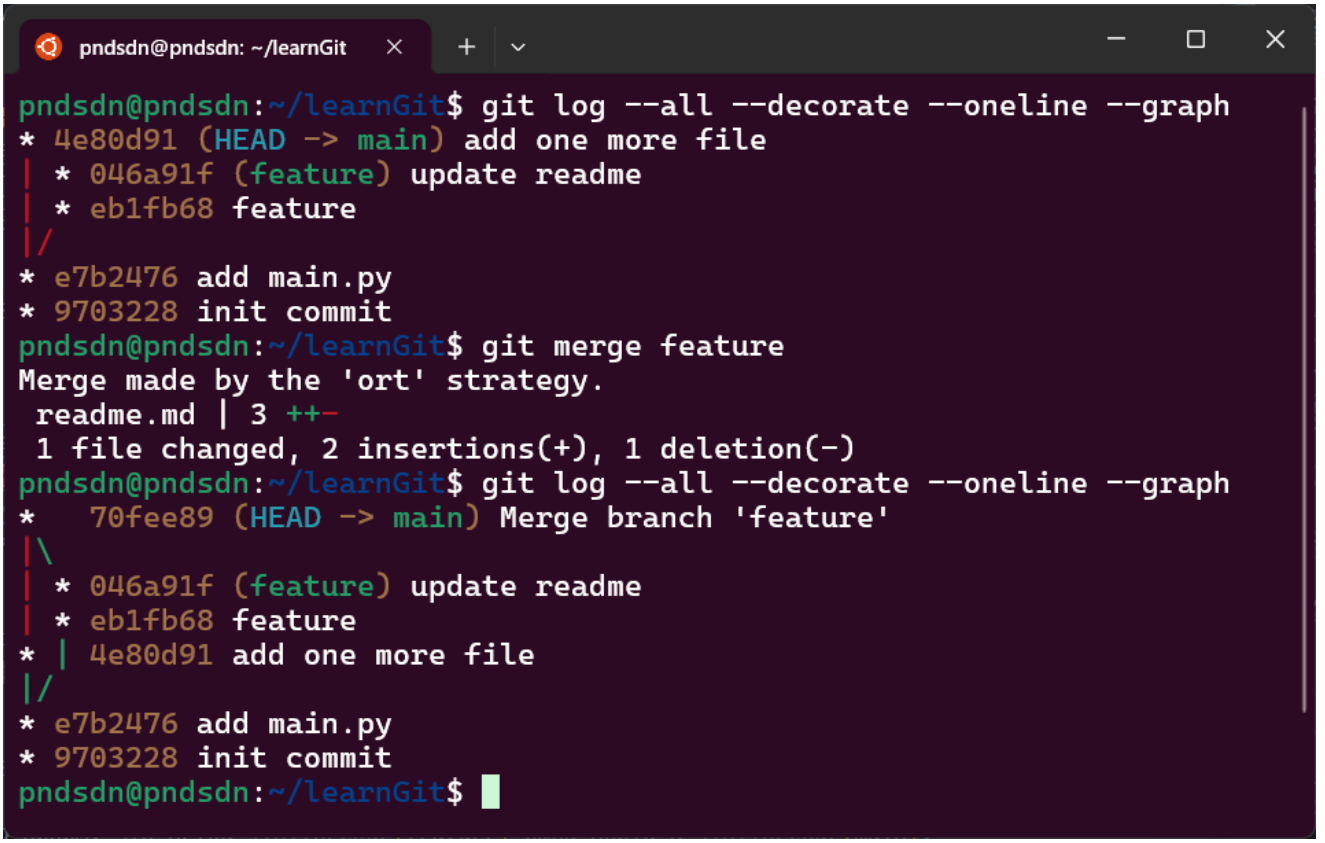


1. Для слияния ветки `<source_branch>` в ветку `<target_branch>` с помощью команды `git merge <source_branch>` необходимо находится на `<target_branch>` . Так как мы хотим влить ветку *feature* в *main*, нам надо переключиться на ветку *main*;
2. Выполним команду `git merge feature` . Обратите внимание на сообщение “Fast-forward“в выводе; В случаях, когда при fast-forward слиянии необходимо оставить историю, можно к команде `git merge <source_branch>` добавить флаг `--no-ff` (no fast-forward), тогда будет создан новый коммит.

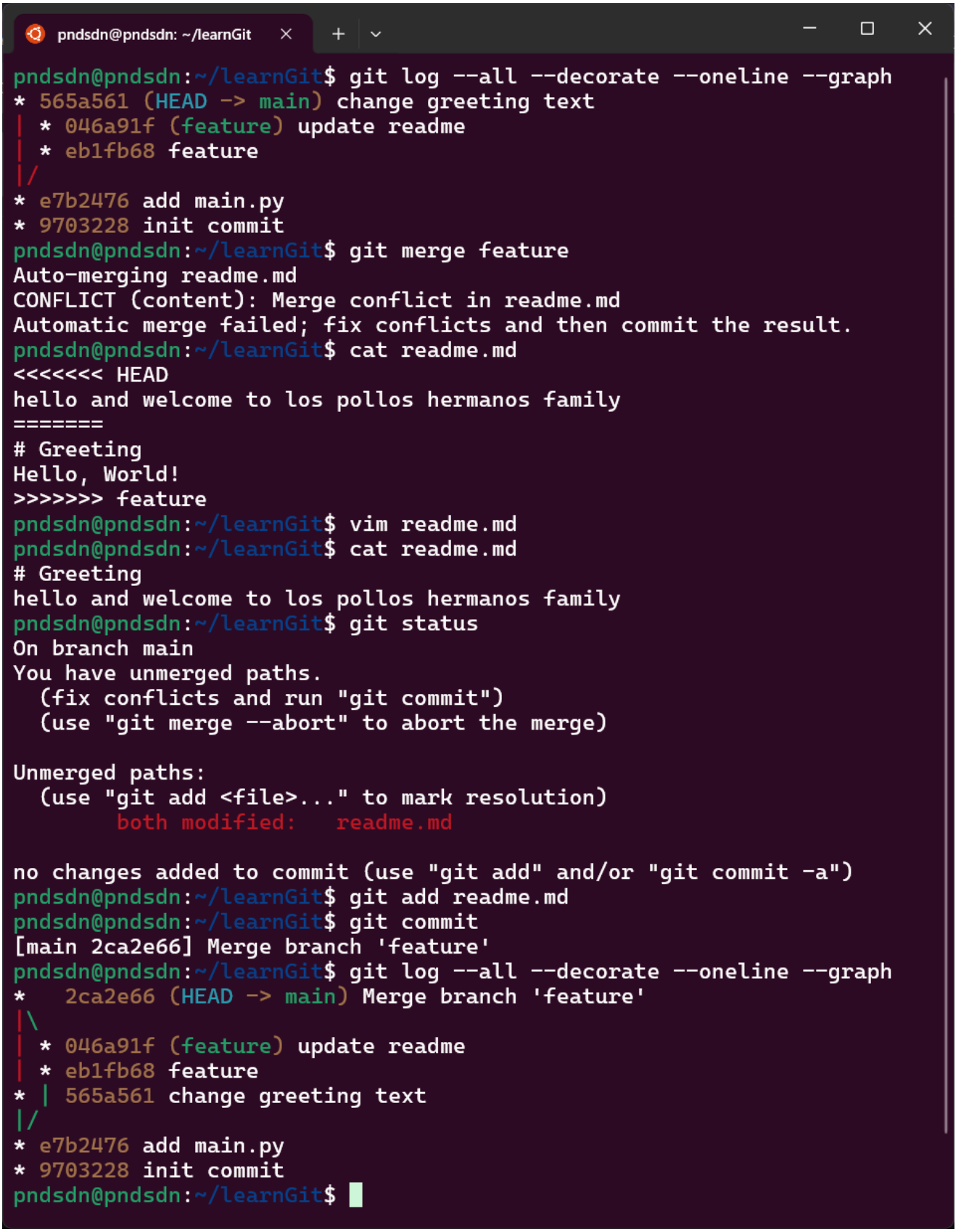


Merge

В ситуации, когда коммит, на котором мы находимся, не является прямым родителем ветки, с которой мы выполняем слияние, Git выполняет простое трёхстороннее слияние, используя последние коммиты объединяемых веток и общего для них родительского коммита.



В данном примере в ветку *main* был добавлен новый файл, которого нет в ветке *feature*. Следовательно, при слиянии Git объединил версии *feature* и *main* в один новый коммит. Но в процессе разработки, один и тот же файл может независимо изменяться в разных ветках, и тогда при слиянии появится конфликт версий, который нам придется решить вручную, и после закончить слияние:



В примере выше файл `readme.md` был изменён в двух ветках. При попытке их слияния Git выдал сообщение о конфликте. При этом файл внутри себя содержит информацию об изменениях текущей ветки (строки между `<<<<<<< HEAD` и `=======`) и вливаемой ветки (строки между `=======` и `>>>>>>> feature`).

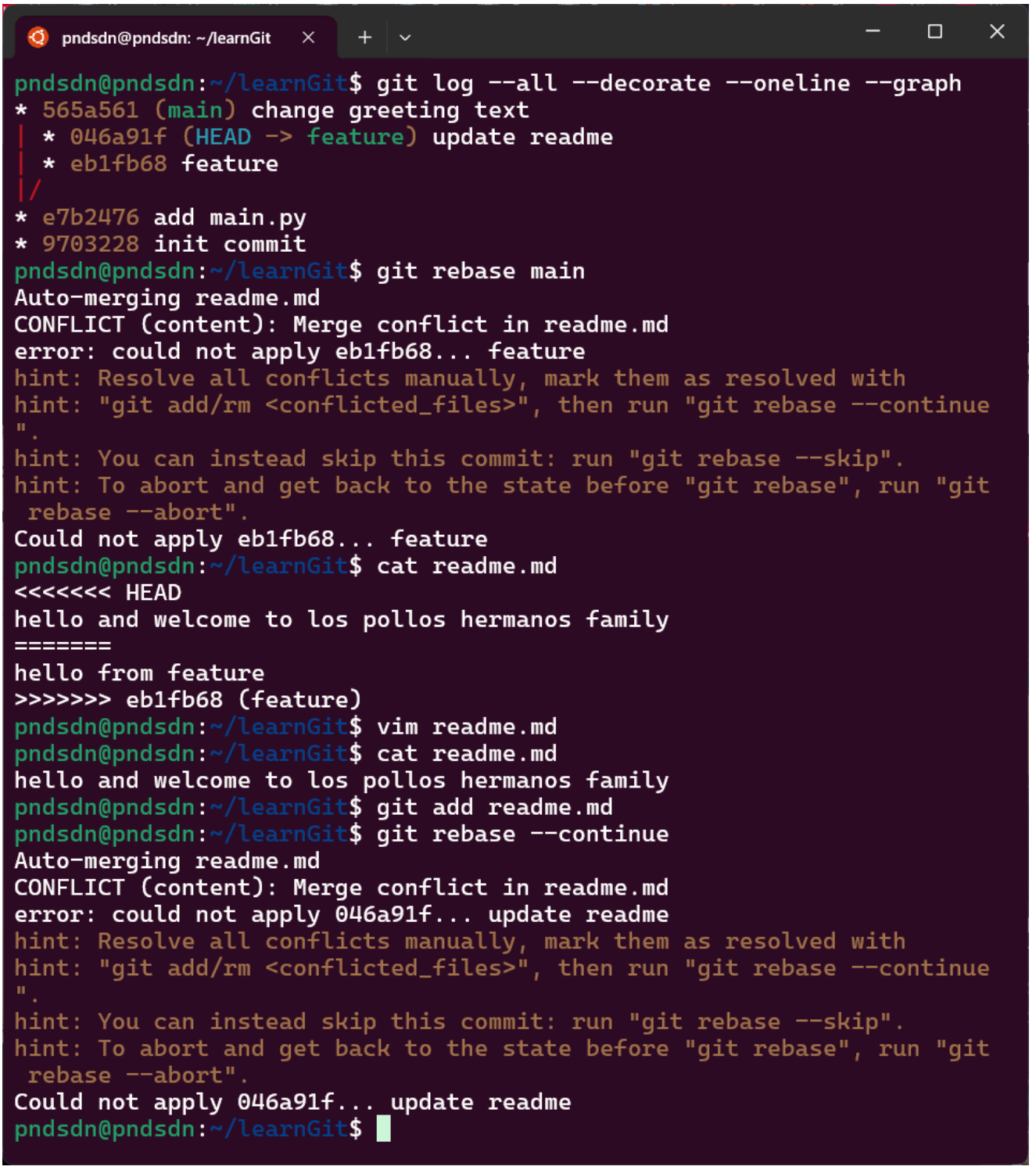
Предположим, что мы хотим оставить строку `# Greeting`, но поменять приветствие на то, которое было сделано в ветке *main*. Отредактируем файл соответствующим образом.

При проверке статуса, Git сообщает, что имеются неслитые изменения. Добавляем изменённый файл в индекс и коммитим.

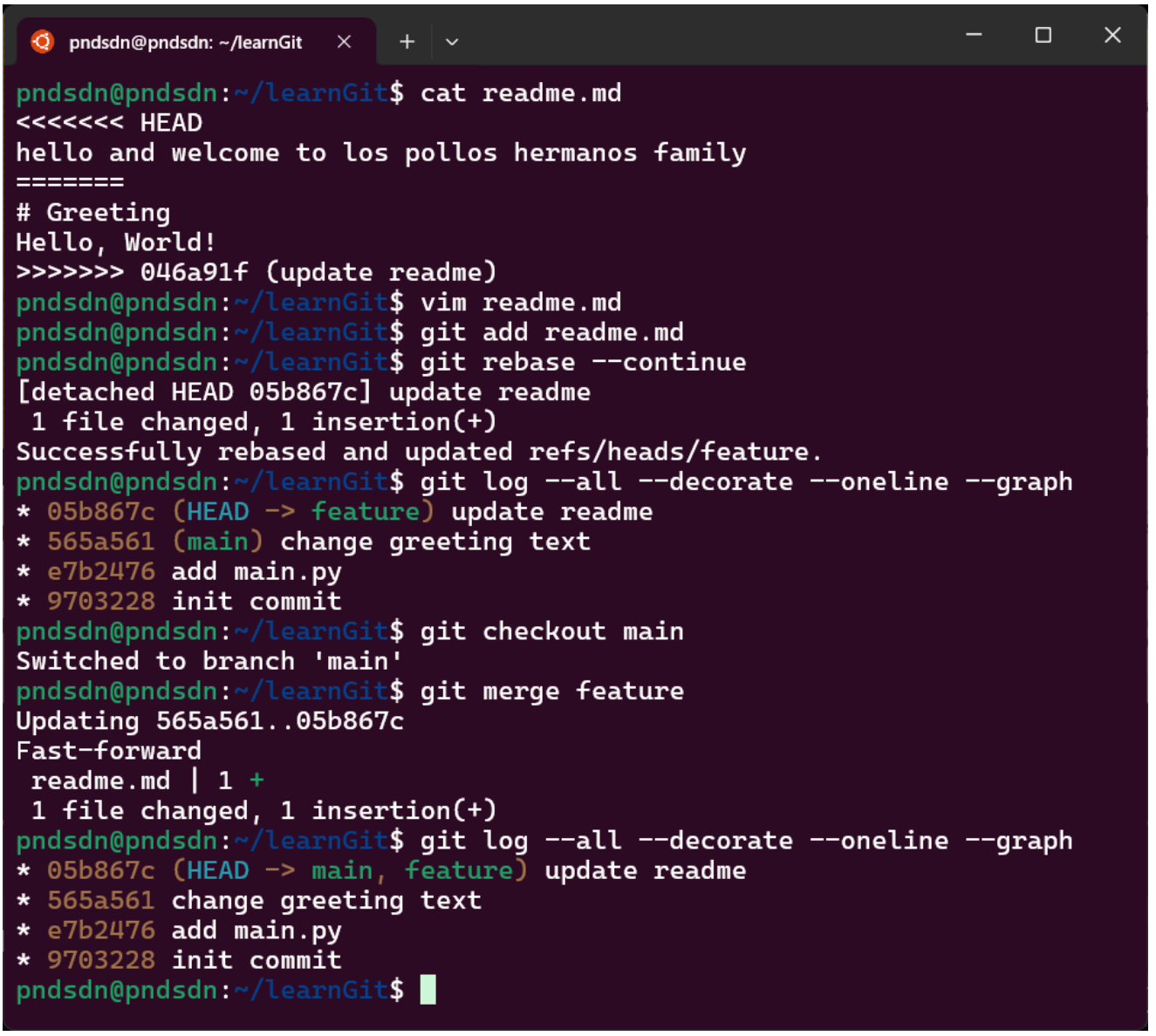
Rebase

Суть метода **rebase** (перебазирования) заключается в том, чтобы изменить родительский коммит текущей ветки. В таком случае берётся общий родительский коммит двух веток (текущей, и той, последний коммит которой станет новой точкой начала ветки). После чего определяется дельта каждого коммита текущей ветки от родительского коммита и сохраняется во временный файл, текущая ветка устанавливается на последний коммит ветки, поверх которой вы выполняете перебазирование, а затем по очереди применяются дельты из временных файлов.

В отличие от команды `git merge`, для `git rabase` необходимо находиться на той ветке, которая будет изменена.



При перебазировании ветки у нас возникает конфликт. Смотрим содержимое и видим, что это конфликт с коммитом `eb1fb68` – всё верно, так как было сказано ранее: при перебазировании определяются изменения для каждого коммита ветки. Решаем конфликт и продолжаем процедуру (`git rebase --continue`) и снова встречаем конфликт, но уже коммита `046a91f` :



Решаем конфликт и продолжаем перебазирование (`git rebase --continue`). После завершения переноса коммитов граф принимает линейный вид. Теперь мы можем переключиться на ветку main и выполнить быстрое перемещение указателя. Обратите внимание, что “перенесённые” коммиты имеют отличный от оригинала хеш. Всё по тому, что это новые коммиты, а исходные попали в состояние, когда ни одна ветка не указывает на них. Такие подвешенные объекты могут оставаться в папке `objects` до очистки.

Синхронизация репозиториев

Git — распределённая СКВ, следовательно, каждый репозиторий является самодостаточным. Чтобы организовать совместную работу, репозитории каждого отдельного участника должны быть синхронизированны. Чтобы настроить связь между двумя репозиториями можно использовать два подхода:

1. Клонирование существующего репозитория (стандартный подход). Команда `git clone <repo-path>`;
2. Настройка связи с существующим репозиторием (продвинутый вариант). Команда `git remote add <code-name> <repo-path>`. Стандартным кодовым именем для единого удалённого репозитория является `origin`.

После настройки связи между репозиториями можно использовать следующие команды:

- `git fetch` — Получение из удалённого репозитория его `objects` и `refs`. Локальные ветки не изменяются.
- `git pull` — Выполняет `fetch` после чего производит интеграцию с текущей активной веткой. Что означает эта интеграция? Git выполняет слияние ассоциированной удалённой ветки с локальной, применяя изменения из неё.
- `git push` — Выполняет отправку локальных `objects` и `refs` в удалённый репозиторий для текущей ассоциированной ветки.

> **Важный** момент при синхронизации репозиториев:
>
> Локальная и удалённая истории должны быть согласованы, чтобы можно было выполнять операции `pull` и `push`. Если история не согласована, git выведет ошибку. Согласование истории можно выполнить локально: решить конфликты или отбросить ненужную историю. Последнее действие считается опасным и обычно запрещено на таких платформах как GitHub, GitLab и других.

Также в Git существует специальный тип репозиториев: Bare репозиторий. Данный тип репозитория фактически состоит только из содержимого папки `.git`. Предназначение данного репозитория только хранить нашу историю и синхронизироваться с другими репозиториями. Для создания такого типа репозитория используется команда `git init` с флагом `--bare`.

Ход работы

Для выполнения данной лабораторной работы требуется Linux-подобная ОС.

Настройте окружение согласно одного из вариантов:

1. Основная ОС или Dual-boot, дистрибутив на базе Linux (Ubuntu и др.) или SELinux (Red Hat и др.).
2. Компонент WSL2 для Windows 10/11 Pro
3. MacOS (несмотря на допустимость этого варианта, MacOS развивалась в другой ветке UNIX-подобных систем (BSD), могут быть сложности с этим вариантом)
4. Виртуальная система через гипервизор (например, Oracle VirtualBox)

Изучение работы с одиночным репозиторием

Создайте два репозитория `repo` и `master`. Для создания `master` используйте флаг `--bare`. Чем отличается **bare** репозиторий от обычного?

Установите имя пользователя на своё и создайте несколько коммитов в репозитории `repo`.

Соедините `repo` и `master` командой:

```
git remote add origin "../master"
```

Отправьте коммиты из репозитория `repo` в репозиторий `master`.

Изучение командной работы с репозиторием

Для выполнения этой части задания объединитесь с другим человеком из вашей группы.

Зайдите на площадку хранения репозиториев (GitFlic, GitLab, GitHub, etc.) и создайте общий репозиторий и проинициализируйте его файлом README.md

(через настройки создания репозиториев). Вы и Ваш одногруппник должны быть добавлены в репозиторий с ролью `maintainer`.

Склонируйте репозиторий локально.

Внесите уникальные изменения в файл `README.md`, закоммите и запуште.

У одного из вас возникнет ошибка при попытке осуществить пуш, синхронизируйте свой репозиторий с удалённым, решите конфликт и попробуйте повторить конфликтную ситуацию для второго человека.

Найдите ответ на вопрос, зачем нужны роли участников на площадке.

Контрольные вопросы

1. Что такое Git?
2. Что такое ветка?
3. Что такое тег?
4. Объясните метод объединения веток fast-forward.
5. Объясните метод объединения веток merge. Какие стратегии работы merge существуют?
6. Зачем нужен rebase?
7. Как git поддерживает командную разработку? Какова роль площадки хранения репозиториев в командной разработке?